

LogPulse AI - Incident Report

Generated : 2026-05-30 21:05:18 UTC

Trace ID : df0c4e82-79a8-4423-85d5-0c60cba97225

1 - Incident Summary

Incident ID : df0c4e82
Category : Unknown
Severity : SeverityLevel.INFO
Source : Fallback
Cache : MISS

2 - Raw Log

```
2026-05-30 18:22:11 ERROR kubelet: Pod worker-api-7d9f8b-xk2q in namespace production OOMKilled (container=worker-api) exceeded 256Mi limit; kubectl describe deployment worker-api -n production shows crashLoopBackOff; kubectl top pod worker-api -n production high memory usage; kubectl get events -n production reports MemoryPressure on node worker-3
```

3 - Classification

Category : Unknown
Severity : SeverityLevel.INFO
Summary : Fallback classification triggered due to inference failure.
Source : Fallback

4 - Diagnosis

Root Cause : Based on the matched playbook 'Pods OOMKilled due to insufficient memory', the likely root cause is a Unknown incident matching the pattern: Fallback classification triggered due to inference failure..

Confidence : 60%

Reasoning : Gemini unavailable; diagnosis derived from playbook 'Pods OOMKilled due to insufficient memory' (RAG confidence=0.7939).

Source Tier : playbook_fallback

Used History : No

Used Playbook : Yes

5 - Proposed Solution

Explanation:

Gemini unavailable; remediation derived from playbook 'Pods OOMKilled due to insufficient memory' (RAG

confidence=0.7939). Review the playbook steps and adapt commands to the live environment before execution.

Steps:

- 1. Identify the specific pods and deployments experiencing OOMKills using `kubectl get events` and `kubectl describe pod <pod-name>`.
- 2. Check the application logs within the affected pods for memory-related errors.
- 3. Review the `memory.limit` and `memory.request` defined in the pod's container spec.
- 4. If the application is known to have increased memory requirements, increment the `memory.limit` for the affected deployment in its YAML.
- 5. If a memory leak is suspected, collaborate with the development team to analyze heap dumps or application-specific memory metrics.
- 6. Consider scaling out the deployment or the node pool if node-wide memory pressure is observed consistently.
- 7. name: my-container

Commands:

```
$ kubectl apply -f deployment.yaml
$ kubectl rollout status deployment/my-service -w
```

6 - Risk Assessment

Risk Level : RISKLEVEL.SAFE

Safe Commands (2):

- [OK] `kubectl apply -f deployment.yaml`
- [OK] `kubectl rollout status deployment/my-service -w`

Flagged Commands (0):

(none)

7 - Operator Decision

Decision : OPERATORDECISION.APPROVED

Executed At : 2026-05-30T18:05:17.884164+00:00

Sandbox Results:

Command : `kubectl apply -f deployment.yaml`

Exit : -1

Time : 0ms

Output:

```
[sandbox unavailable -- docker not found]
```

Command : `kubectl rollout status deployment/my-service -w`

Exit : -1

Time : 0ms

Output:

```
[sandbox unavailable -- docker not found]
```

8 - Historical Context

Times Seen : 1
First occurrence -- no prior incidents.

9 - Incident Timeline

[18:05:17] WORKFLOW_COMPLETED -> Completed in 79071ms